

Project 2: Neural Network

(B252-E162110) Soft Computing Techniques to Characterize Thermal Systems

Šimon Hykl¹
Czech Technical University in Prague

This report covers the neural network developed in the ‘Soft Computing Techniques to Characterize Thermal Systems’ class. The purpose of this project is to predict the heat flux in heat exchangers from mass flow and temperature inputs. The neural network is built in MATLAB and is first tested on a synthetic 1D function. Then it is trained on the heat exchanger data and tested on another dataset.

I. Introduction

The neural network follows the classical structure according to [1] and [2]. First, the neuron values and outputs are computed in a feedforward run. Then, the error is backpropagated from the output layer to the first hidden layer.

The input data are treated individually and the connection weights and neuron biases are updated after each datapoint. Unfortunately, this has a severe negative impact on the computational time. There is, however, no way to vectorize the learning process, as that would lead to averaging the error across the dataset and thus predicting something akin to a moving average filter.

II. Script structure

A. Main training loop

As was mentioned before, the datapoints are evaluated one-by-one during the learning loop. In each loop, the data must be normalized. By design, the neural network cannot handle the data in their original ranges. To avoid saturation of the neuron outputs, the $[-0.85, 0.85]$ range was chosen ($\tanh(x)$ used as the neuron activation function).

```
initializeWeights();
initializeBias();

for i = 1:imax
    for j = 1:size(output, 1)
        input_01 = normalizeData(input(j));
        target_01 = normalizeData(target(j));

        neuralNetworkInference(ann, input_01);
        neuralNetworkBackpropagation(ann, target_01);

        output(j) = denormalizeData(output_01);
    end
end
```

¹Student at Department of Aerospace Engineering, simon.hykl@fs.cvut.cz

B. Inference

During the inference phase, the script loops over the layers and computes the neuron values as a weighted sum of all previous inputs, complemented with the bias. The output of each neuron is determined using the non-linear neuron activation function. The input layer is linear (i.e. the activation function is identity).

```
function [out_01, ann] = neuralNetworkInference(ann, in_01)
    ann.neuron_val{1} = in_01;
    ann.neuron_out{1} = ann.neuron_val{1};

    for i = 2:n_layers
        ann.neuron_val{i} = ann.bias{i} + ...
                            ann.neuron_out{i-1}*ann.w{i-1};
        ann.neuron_out{i} = neuronActivation( ...
                            ann.neuron_val{i});
    end

    out_01 = ann.neuron_out{end};
end
```

C. Backpropagation

During backpropagation, the script loops over the layers again, but this time in reverse order. The delta of the output layer is computed as the straightforward error (between ANN output and target) multiplied by the derivation of the activation function. Then, the deltas for all other layers, except the input layer, are obtained using the previously computed deltas, connection weights and neuron output values. The deltas for the input layer are irrelevant.

In a second loop the weight and bias corrections are computed and applied.

The backpropagation is very sensitive to correct indexing and great care must be taken to properly mind the layer indices and array sizes.

```
function ann = neuralNetworkBackpropagation(ann, target_01)
    ann.delta{end} = (target_01 - ann.neuron_out{end})* ...
                    neuronActivationDerivative(ann.neuron_out{end});

    for i = (n_layers-1):-1:2
        ann.delta{i} = (ann.delta{i+1}*ann.w{i}.').* ...
                    neuronActivationDerivative(ann.neuron_out{i});
    end

    for i = (n_layers-1):-1:1
        ann.dw{i} = learning_rate.* ...
                    ann.neuron_out{i}.*ann.delta{i+1};
        ann.w{i} = ann.w{i} + ann.dw{i};

        ann.db{i+1} = learning_rate.*ann.delta{i+1};
        ann.bias{i+1} = ann.bias{i+1} + ann.db{i+1};
    end
end
```

D. Neuron activation function

The neuron activation function can be chosen quite arbitrarily. In this project, the hyperbolic tangent was chosen. It is given simply as

$$f(x) = \tanh(x) \quad (1)$$

and its derivative is

$$f'(x) = 1 - \tanh^2(x) = 1 - f^2(x) \quad (2)$$

All suitable activation functions have a common behavior: they are non-linear (piecewise-linear at the very least) and their outputs are bounded to a finite interval. The asymptotic behavior at large inputs is exactly what causes the neuron output saturation if additional preventive measures are not taken (i.e. data normalization to a smaller range).

E. Initialization of weights and biases

The behavior of the neural network is also sensitive to the initialization of the connection weights. The weights must be initialized randomly, otherwise the backpropagation will produce uniform updates. The biases may be initialized to zero.

```
function w = initializeWeights()
    for i = 1:(n_layers-1)
        w{i} = randn(n_neuron(i), n_neuron(i+1)).* ...
            sqrt(1/n_neuron(i))
    end
end

function b = initializeBias()
    for i = 1:n_layers
        bias{i} = zeros(1, n_neuron(i));
    end
end
```

III. Neural network performance

The neural network was tested on a synthetic 1D function given as

$$f(x) = 1 + 2x + 3x^2 + e^x - \cos(2\pi x) \quad (3)$$

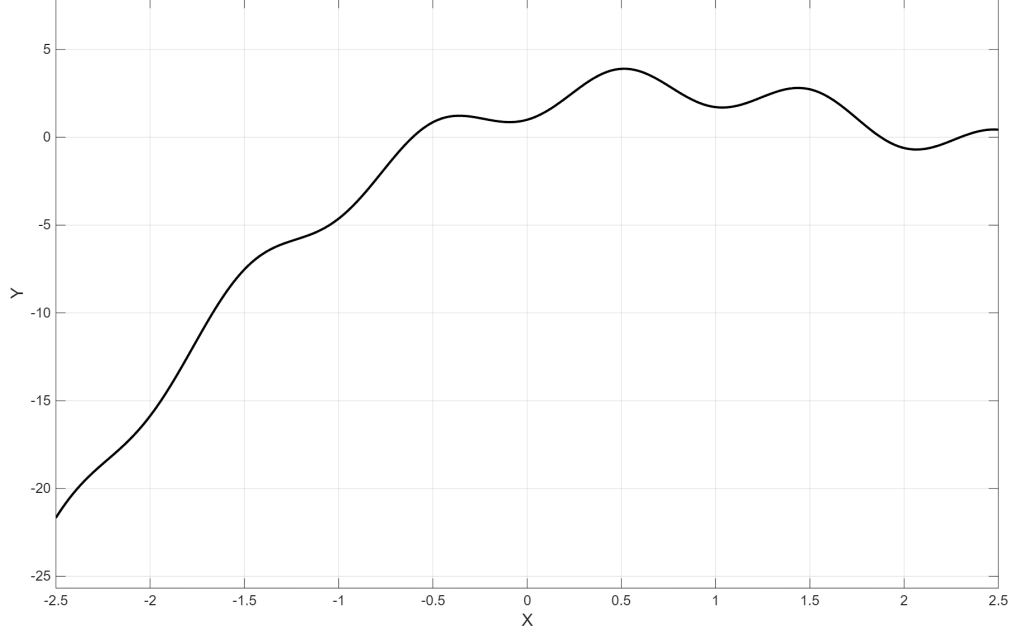


Fig. 1 Target function

The function determines the number of input and output neurons. For this case, 2 hidden layers were chosen with 5 and 3 neurons respectively.

To determine the optimal learning rate, a sweep was performed over values $\lambda \in \{0.01, 0.1, 0.4, 0.6, 0.8\}$. The error convergence for each learning rate value is depicted on figure 2.

Another sweep was performed over different architectures (see figure 3), exploring different numbers of neurons and even 1 case with more hidden layers. Based on this sweep, the $[1, 5, 3, 1]$ architecture was chosen. Although the architecture with 3 hidden layers yields better results, it is prone to oscillations.

The maximum training time was set to $5 \cdot 10^5$ iterations (epochs). Up until 10^3 iterations, the network performs irrespective of the learning rate. After that, higher learning rate values seems to yield more accurate results. Interestingly, even with $\lambda = 0.8$, the network is mostly stable. Oscillations can be observed, but given enough time, they converge to a very low error value. Values over 0.8 were tested but not included in figure 2 for legibility reasons. Any value of λ greater than 1.3 diverged.

For further evaluation, $\lambda = 0.4$ was chosen as it provides error comparable to $\lambda = 0.8$ but is stable.

The neural network was trained on the synthetic function in range $[-1, 1]$. On figure 4 prediction by the trained network is compared against the true target function and a polynomial fit. The graph perfectly illustrates the network's different behavior inside and outside of the training range. Inside, the network can predict the functional values very well, even better than the polynomial. Outside the range, however, it fails to extrapolate the correct values (according to expectations). The neural network does have one great advantage over the polynomial fit. That is, it is bounded. Outside of the training range, it doesn't diverge, but rather keeps a constant value.

Figure 5 showcases the network's applicability to real-world problems. The input and target data were provided by Zhao [3]. This dataset is much larger than in the previous case. Therefore, to decrease the computational time, the maximum number of training iterations was reduced to 10^5 . The graph shows a very good fit of the ANN prediction to the target data. This proves that the developed neural network is capable of handling general data with a general number of inputs and outputs.

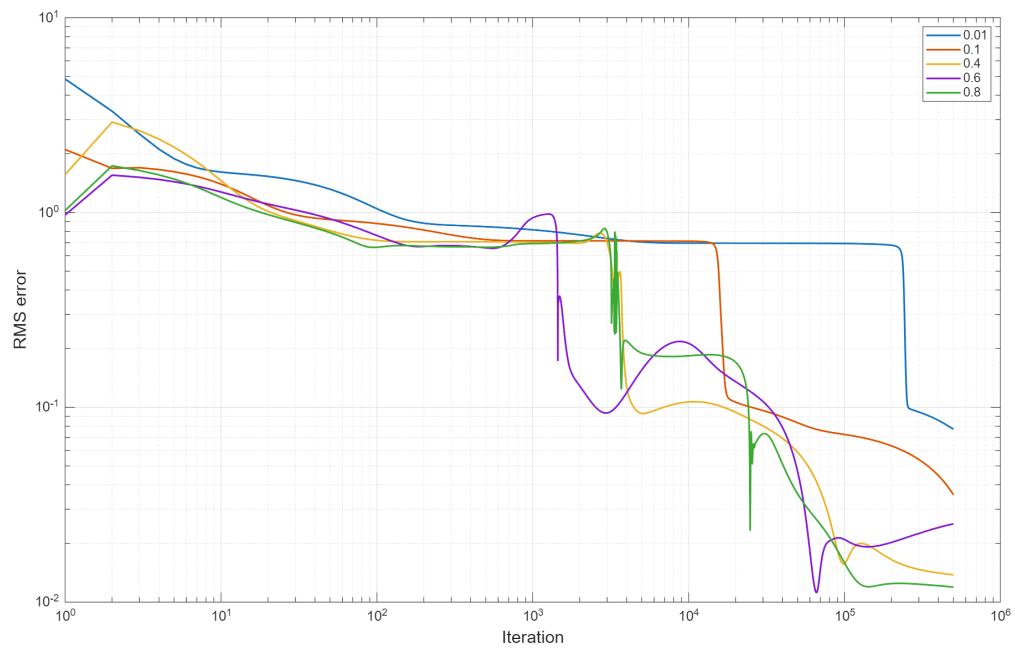


Fig. 2 ANN prediction error based on learning rate

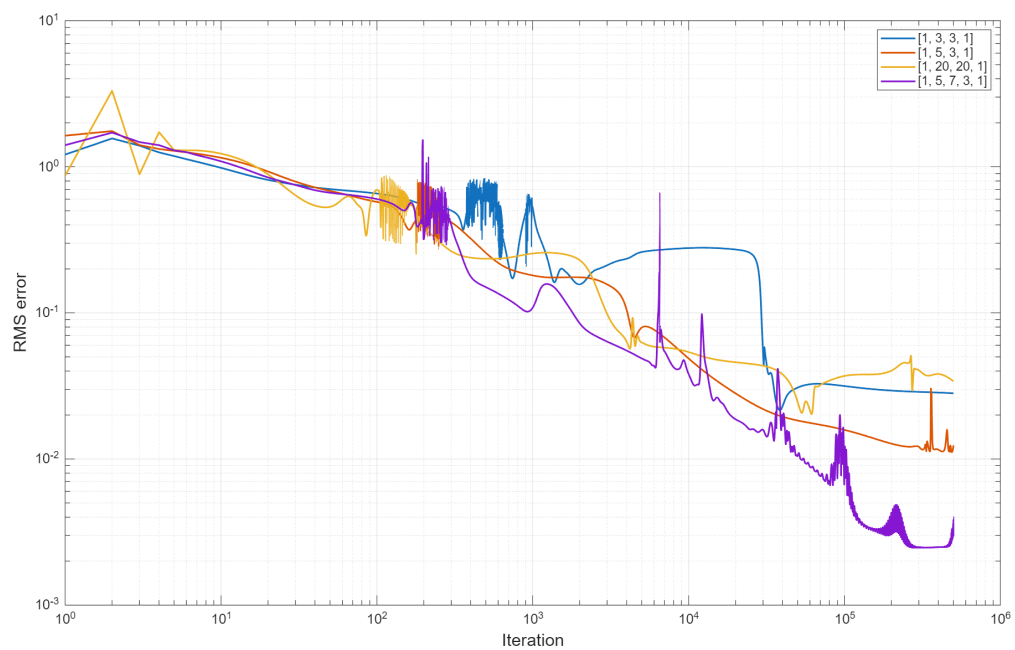


Fig. 3 ANN prediction error based on architecture

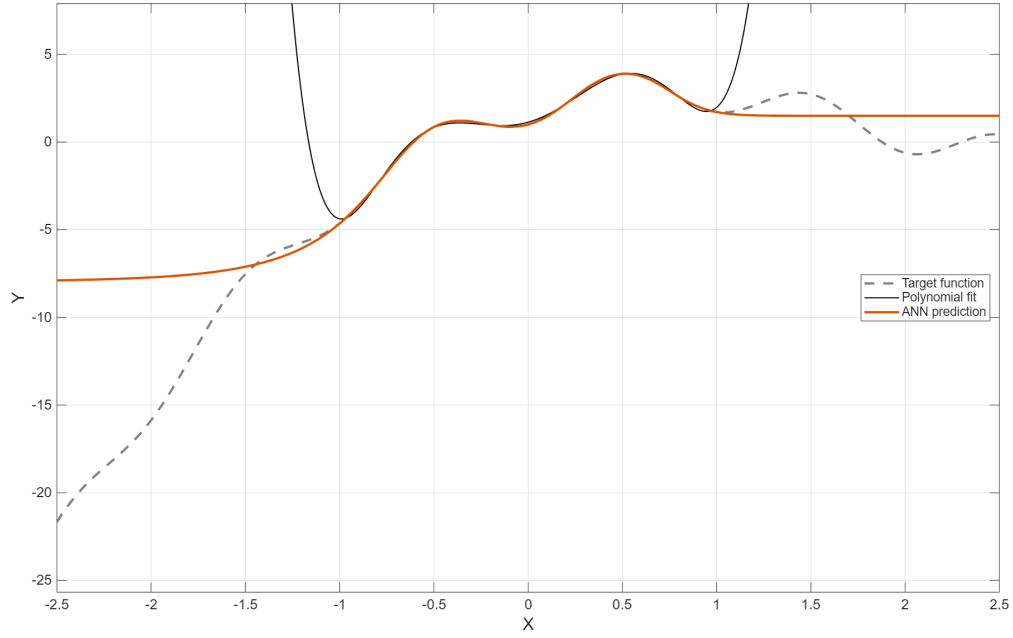


Fig. 4 ANN prediction of the target function

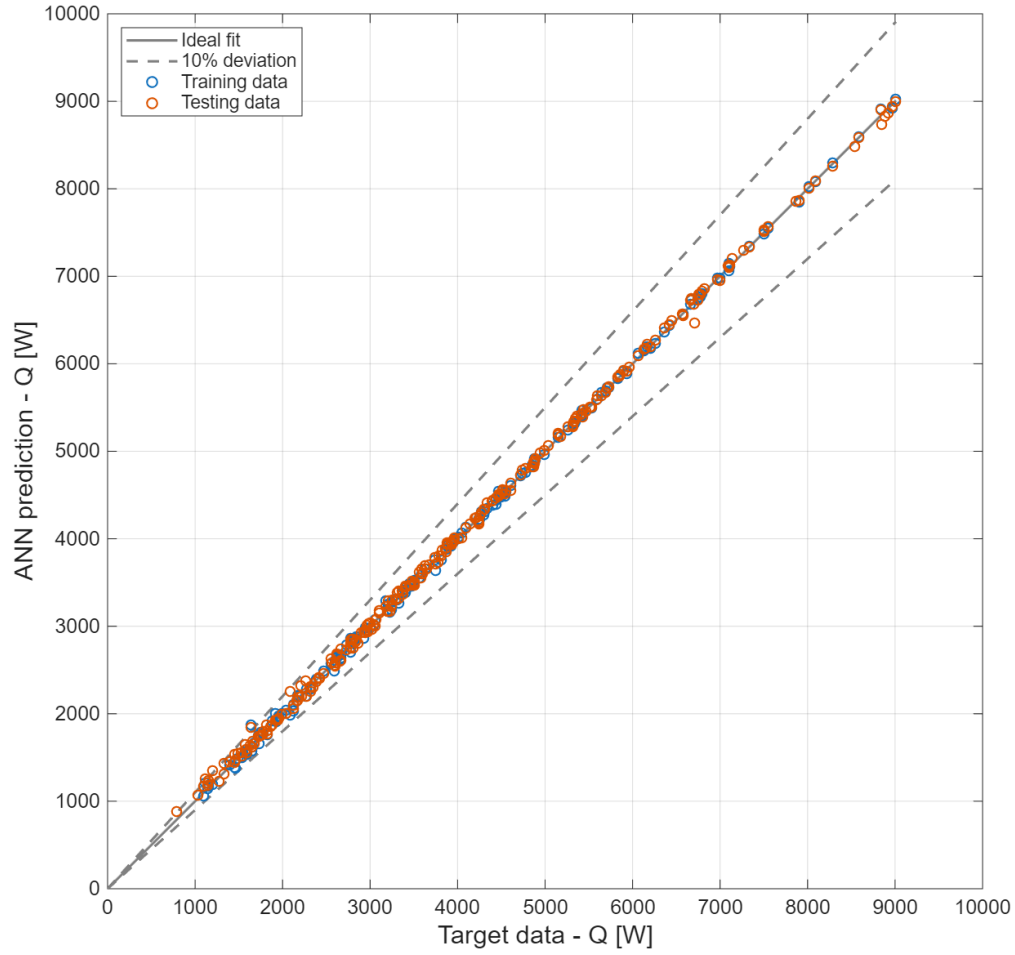


Fig. 5 ANN prediction of heat exchanger data

IV. Conclusion

An artificial neural network was implemented in MATLAB. The network uses the hyperbolic tangent as the neuron activation function and is developed to be largely modular. The network architecture and learning rate can be freely changed. The network was first trained and tested on a sythetic 1D function to prove the functionality of the implementation. Afterwards, real-world applicability was proved on a real heat exchanger dataset.

References

- [1] Chhabra, R. P., editor, *CRC handbook of thermal engineering*, Mechanical and Aerospace Engineering Series, CRC Press,, Boca Raton, FL, second edition. ed., 2018, Includes bibliographical references and index.
- [2] Díaz, G., Sen, M., Yang, K. T., and McClain, R. L., “Simulation of Heat Exchanger Performance by Artificial Neural Networks,” *HVAC&R Research*, Vol. 5, No. 3, July 1999, pp. 195–208.
- [3] Zhao, X., “Performance of a single-row heat exchanger at low in-tube flow rates,” MS Thesis, Universityof Notre Dame, Notre Dame, IN, 1995.